

Proyecto Integrador: Atenea.io

Historias de Usuario — MPA con Escalabilidad Vertical y Gamificación

Contexto del Proyecto

Una institución educativa requiere modernizar su gestión académica mediante una **Multiple Page Application (MPA)** construida en JavaScript puro (Vanilla JS / Vite). La plataforma, denominada **Atenea.io**, integra autenticación por roles, CRUD de notas, gestión de cursos y horarios, comunicados institucionales y un **sistema de gamificación multi-tipo**. El diseño técnico contempla **escalabilidad vertical**: la arquitectura debe crecer en capacidad (más usuarios, más datos, más módulos) sin rediseñar la base.

Roles del sistema: Rector (admin) · Profesor (teacher) · Estudiante (user).

Principios de Escalabilidad Vertical aplicados

- Paginación server-side en todos los listados (?_page=&_limit=) para no saturar el cliente.
- Cálculos agregados (promedios, leaderboard) en el servidor, nunca en cliente.
- Esquema de datos extensible: nuevos tipos de punto, roles o módulos se añaden sin modificar estructuras existentes.
- Router y módulos desacoplados: cada feature es un plugin que se registra, no una modificación del núcleo.
- Separación estricta de capas: UI / Servicios / Store / Router facilita escalar cada capa independientemente.

Autenticación y Sesión

US-01 Prioridad: Alta

Como usuario registrado, **quiero** iniciar sesión con mi correo y contraseña en la plataforma Atenea.io, **para** acceder únicamente a las funciones y datos que corresponden a mi rol.

Criterios de aceptación

- El formulario valida que el correo tenga formato válido y la contraseña no esté vacía.
- Las credenciales se verifican contra el endpoint /users de json-server (o API externa).
- Tras inicio exitoso, se redirige automáticamente al dashboard del rol correspondiente (rector / profesor / estudiante).
- Si las credenciales son incorrectas, se muestra un mensaje de error descriptivo sin revelar cuál campo falló.
- La sesión queda almacenada en localStorage (opción "recuérdame") o sessionStorage (sesión de pestaña).

🚫 Restricciones

- Un usuario no puede acceder a módulos de otro rol.
- Tras 3 intentos fallidos consecutivos, el botón de login se deshabilita 30 s.

Escalabilidad vertical: El módulo de autenticación debe ser stateless para permitir replicación horizontal del frontend sin perder sesiones.

Nota técnica: Implementar como página independiente (MPA): /login.html. El router redirige aquí cuando no hay sesión activa.

US-02 Prioridad: Alta

Como usuario autenticado, **quiero** mantener mi sesión activa al refrescar o navegar entre páginas del MPA, **para** no interrumpir mi flujo de trabajo por recargas accidentales.

Criterios de aceptación

- Al refrescar cualquier página, la sesión se recupera desde localStorage/sessionStorage automáticamente.
- El router verifica el token/sesión antes de renderizar cualquier módulo protegido.
- Si la sesión expiró, se redirige a /login.html con un mensaje informativo.
- El logout limpia completamente localStorage y sessionStorage y redirige a /login.html.

Escalabilidad vertical: Diseñar el guard como función pura y reutilizable para facilitar la adición de nuevos roles sin modificar la lógica central.

Nota técnica: Usar un guard/middleware de rutas en el router del MPA que se ejecute antes de cada cambio de página.

Rector (Admin)

US-03 Prioridad: Alta

Como rector, **quiero** gestionar comunicados y anuncios institucionales (crear, editar, eliminar), **para** mantener informada a toda la comunidad educativa de manera centralizada y oportuna.

Criterios de aceptación

- El rector puede crear un comunicado con título, cuerpo, fecha de publicación y destinatario (todos / profesores / estudiantes).
- Los comunicados aparecen en el dashboard del rol destinatario en orden cronológico inverso.
- El rector puede editar o eliminar un comunicado existente; los cambios se reflejan en tiempo real para los receptores.
- Cada comunicado queda registrado con fecha, autor y estado (Publicado / Borrador).

Escalabilidad vertical: La tabla de comunicados debe soportar paginación del lado del servidor para mantener rendimiento con miles de registros.

Nota técnica: CRUD sobre /comunicados en json-server. Filtrar por rol destino en el GET del dashboard.

US-04 Prioridad: Alta

Como rector, **quiero** crear, editar, desactivar y consultar cursos con sus horarios y cupos, **para** organizar y actualizar la oferta académica sin depender de registros manuales en papel.

Criterios de aceptación

- Cada curso contiene: ID, nombre/grado, materia, profesor asignado, horario (día y hora), cupo máximo y estado (Activo/Inactivo).
- El rector puede asignar o reasignar un profesor a un curso desde un selector de usuarios con rol "teacher".
- Al desactivar un curso, los estudiantes matriculados reciben una notificación en su dashboard.
- El rector puede cambiar el horario de un curso; el cambio se propaga al calendario del profesor y los estudiantes matriculados.
- No se pueden crear dos cursos con el mismo profesor en el mismo horario.

⛔ Restricciones

- Eliminar un curso con notas registradas está bloqueado; solo se puede desactivar.

Escalabilidad vertical: El modelo de cursos debe admitir múltiples sedes sin cambios en la estructura de datos (añadir campo "sedeld").

Nota técnica: CRUD /cursos. Validar solapamiento de horarios antes del POST/PATCH.

US-05 Prioridad: Alta

Como rector, **quiero** gestionar el registro completo de estudiantes (alta, edición, asignación a cursos y baja), **para** mantener actualizado el padrón académico y garantizar que cada estudiante esté correctamente matriculado.

Criterios de aceptación

- El rector puede dar de alta un nuevo estudiante con nombre, correo, contraseña temporal y asignación inicial de curso.
- El rector puede editar los datos personales y reasignar cursos de cualquier estudiante.
- La baja de un estudiante cambia su estado a "Inactivo"; sus notas e historial se conservan.
- El listado de estudiantes soporta búsqueda por nombre, curso y estado.

Escalabilidad vertical: Implementar paginación server-side (?_page=&_limit=) para escalar a miles de estudiantes sin degradar la carga inicial.

Nota técnica: CRUD /users filtrando por role:"student". La búsqueda usa query params de json-server (?nombre_like=).

US-06 Prioridad: Alta

Como rector, **quiero** gestionar el registro de profesores y sus asignaciones de cursos, **para** garantizar cobertura docente completa y evitar conflictos de asignación.

Criterios de aceptación

- El rector puede crear, editar y desactivar cuentas de profesores.

- Desde el perfil del profesor se visualizan todos los cursos asignados.
- No se puede desactivar un profesor con cursos activos sin reasignar esos cursos primero.

Nota técnica: *CRUD /users filtrando role:"teacher". Validar dependencias en DELETE/PATCH.*

US-07 Prioridad: **Media**

Como rector, **quiero** consultar un dashboard con estadísticas e indicadores académicos globales, **para** tomar decisiones informadas sobre el desempeño institucional sin requerir reportes manuales.

Criterios de aceptación

- El dashboard muestra: promedio general de notas por curso y periodo, número de estudiantes activos/inactivos, top 5 estudiantes por puntos de gamificación.
- Los indicadores se recalculan al entrar al módulo (GET fresco a la API).
- Permite filtrar por período académico.

Escalabilidad vertical: *Los cálculos de estadísticas deben realizarse en el servidor (json-server + middlewares o API externa) para no saturar el cliente cuando el volumen de datos crezca.*

US-08 Prioridad: **Media**

Como rector, **quiero** configurar y ampliar los tipos de punto del sistema de gamificación, **para** adaptar la motivación académica a distintos objetivos pedagógicos sin romper el sistema existente.

Criterios de aceptación

- El rector puede crear un nuevo tipo de punto con nombre, ícono/color y descripción.
- Al agregar un nuevo tipo, la tabla de clasificación global se actualiza automáticamente para incluirlo.
- Un tipo de punto puede desactivarse; los puntos ya asignados de ese tipo se conservan.
- El sistema nunca limita a una sola "moneda": puede coexistir Puntos de Excelencia, Puntos de Actitud, Tokens de Participación, etc.

Restricciones

- Solo el rector puede crear o desactivar tipos de punto.

Escalabilidad vertical: *El esquema de tipos de punto debe ser ampliable sin migraciones destructivas: añadir un nuevo tipo es un simple POST, no un ALTER TABLE.*

Nota técnica: *CRUD /pointTypes. La tabla de puntos /studentPoints referencia pointTypeId, nunca un campo hardcodedo.*

Profesor (Teacher)

US-09 Prioridad: **Alta**

Como profesor, **quiero** ver el listado de los cursos que tengo asignados y acceder al detalle de cada uno, **para** planificar mi trabajo docente y acceder rápidamente a mis estudiantes y materias.

Criterios de aceptación

- El dashboard del profesor muestra solo los cursos con su ID como profesor asignado.
- Cada tarjeta de curso muestra: materia, grado, horario, número de estudiantes matriculados.
- Al hacer clic en un curso se abre su página de detalle con lista de estudiantes y acceso a la gestión de notas.

Nota técnica: *GET /cursos?profesorId={userId}. Página independiente /cursos-profesor.html en el MPA.*

US-10 Prioridad: Alta

Como profesor, **quiero** registrar, editar, publicar y eliminar notas de los estudiantes de mis cursos, **para** mantener el boletín académico actualizado y controlar cuándo el estudiante puede ver sus calificaciones.

Criterios de aceptación

- El profesor puede crear una nota con: estudiante, materia, período, valor numérico y desempeño calculado automáticamente (Insuficiente <3.0 / Mínimo 3.0–3.9 / Satisfactorio 4.0–4.4 / Avanzado 4.5–5.0).
- Una nota en estado "Borrador" no es visible para el estudiante.
- Al publicar una nota (cambiar estado a "Publicada"), el boletín del estudiante se actualiza automáticamente.
- El profesor solo ve y gestiona las notas de los cursos que tiene asignados.
- Se puede eliminar una nota propia solo si está en estado "Borrador".

Restricciones

- El rector puede leer cualquier nota pero no crearlas ni editarlas.
- Los estudiantes no pueden ver notas en Borrador.

Escalabilidad vertical: *El endpoint de notas debe soportar filtros combinados (?estudiantId=&cursoId=&periodo=) para escalar sin cargar datos completos en cliente.*

Nota técnica: *CRUD /notas. Filtrar siempre por cursoId perteneciente al profesor autenticado (validación client-side + API).*

US-11 Prioridad: Alta

Como profesor, **quiero** asignar puntos de gamificación a mis estudiantes por materia, indicando motivo y tipo de punto, **para** reconocer el esfuerzo y logros académicos de manera sistemática y motivar el compromiso estudiantil.

Criterios de aceptación

- El profesor selecciona un estudiante de su curso, el tipo de punto (según configuración del rector) y una cantidad.
- Debe ingresar un motivo descriptivo (ej. "Excelente exposición oral").
- Cada asignación queda registrada con fecha, motivo, tipo de punto, cantidad y profesor que la otorgó.

- El saldo del estudiante se actualiza inmediatamente en la tabla de clasificación.
- Solo el profesor puede otorgar puntos; el rector puede auditar el historial.

⊘ Restricciones

- El estudiante no puede autoasignarse puntos.
- El profesor solo puede asignar puntos en los cursos que tiene asignados.

Escalabilidad vertical: Almacenar los puntos como transacciones individuales (event-sourcing ligero) permite auditoría completa y facilita agregar nuevos tipos sin reescribir el historial.

Nota técnica: `POST /studentPoints {studentId, cursold, pointTypeId, cantidad, motivo, profesorId, fecha}`. La suma de puntos se recalcula en cada consulta del leaderboard.

US-12 Prioridad: Media

Como profesor, **quiero** consultar mi horario de clases y recibir comunicados institucionales del rector, **para** organizar mi jornada y estar informado de las novedades sin depender de canales externos.

Criterios de aceptación

- La sección "Mi Horario" muestra una vista semanal de los cursos asignados, con día y hora.
- Los comunicados del rector aparecen en el dashboard ordenados por fecha descendente.
- Los comunicados no leídos se marcan visualmente.

Estudiante (User)

US-13 Prioridad: Alta

Como estudiante, **quiero** consultar mi boletín de notas por materia y período académico, **para** hacer seguimiento continuo a mi rendimiento sin necesitar intermediarios.

Criterios de aceptación

- El boletín muestra únicamente las notas en estado "Publicada" que corresponden al estudiante autenticado.
- Cada fila muestra: materia, período, nota numérica, desempeño y nombre del profesor.
- El estudiante puede filtrar por período académico.
- El promedio general del período se calcula y muestra al tope del boletín.

⊘ Restricciones

- El estudiante no puede ver las notas de otros estudiantes.
- No puede modificar ninguna nota.

Nota técnica: `GET /notas?estudiantId={userId}&estado=Publicada`. Renderizar en página dedicada `/boletin.html` del MPA.

US-14 Prioridad: Alta

Como estudiante, **quiero** ver los cursos en los que estoy matriculado, sus anuncios y el calendario académico, **para** tener en un solo lugar toda la información relevante de mi actividad académica.

Criterios de aceptación

- El dashboard muestra tarjetas de cada curso matriculado con materia, grado y horario.
- Los anuncios del curso y los comunicados institucionales se muestran en sección separada, ordenados por fecha.
- El calendario muestra las fechas importantes del año académico.

US-15 Prioridad: Alta

Como estudiante, **quiero** ver mi puntuación total y desglosada por tipo de punto y materia, y consultar la tabla de clasificación, **para** conocer mi posición en el sistema de gamificación y motivarme a mejorar mi desempeño.

Criterios de aceptación

- La sección "Mis Puntos" muestra el saldo actual por cada tipo de punto activo.
- Hay una vista desglosada por materia: cuántos puntos obtuvo en cada curso y de qué tipo.
- La tabla de clasificación (leaderboard) muestra el ranking de todos los estudiantes del mismo curso/grado.
- El leaderboard puede filtrarse por tipo de punto o verse de forma global (suma de todos los tipos).
- El estudiante no puede ver el saldo exacto de puntos de otros (solo el ranking).

Escalabilidad vertical: El leaderboard se construye siempre desde el servidor con una consulta agregada, no cargando todos los registros en cliente, para escalar con miles de estudiantes.

US-16 Prioridad: Media

Como estudiante, **quiero** canjear mis puntos en la tienda de recompensas por beneficios académicos o institucionales, **para** obtener reconocimiento tangible de mi esfuerzo y mantener la motivación a largo plazo.

Criterios de aceptación

- La tienda muestra recompensas disponibles con su costo en puntos (por tipo).
- El estudiante solo puede canjear si tiene saldo suficiente del tipo de punto requerido.
- Al realizar el canje, el saldo se descuenta y queda un registro de la transacción.
- El historial de canjes es visible para el estudiante y el rector.

⛔ Restricciones

- Los puntos canjeados no pueden revertirse automáticamente; requieren aprobación del rector.

Escalabilidad vertical: El catálogo de recompensas debe ser configurable por el rector sin cambios de código, para añadir o quitar premios según el año académico.

Nota técnica: POST /canjes {studentId, rewardId, puntosUsados, pointTypeId}. Verificar saldo suficiente antes del commit.

US-17 Prioridad: Baja

Como estudiante, **quiero** ver y editar mi perfil personal (nombre, foto, contraseña), **para** mantener mi información actualizada y personalizar mi experiencia en la plataforma.

Criterios de aceptación

- El estudiante puede cambiar su nombre, foto de perfil y contraseña.
- Cambiar la contraseña requiere confirmar la contraseña actual.
- Los cambios se persisten en /users vía PATCH.

MPA — Arquitectura y Navegación

US-18 Prioridad: Alta

Como usuario, **quiero** navegar entre los módulos de la plataforma sin recargas completas de página, **para** tener una experiencia fluida similar a una app nativa, reduciendo el tiempo de espera.

Criterios de aceptación

- La aplicación implementa un router client-side basado en History API o hash (#).
- Cada cambio de ruta renderiza dinámicamente el módulo correspondiente en el contenedor principal sin recargar el HTML base.
- El botón Atrás/Adelante del navegador funciona correctamente.
- Las rutas protegidas (cualquier módulo distinto a login) redirigen a /login.html si no hay sesión activa.
- Al copiar y pegar una URL directa, la aplicación carga la página correcta o redirige al login.

Escalabilidad vertical: El router debe permitir la adición de nuevas rutas como un simple registro (`router.register("/nueva-ruta", handler)`) sin modificar el núcleo del enrutador.

Nota técnica: Implementar un Router class con métodos `navigate(route)`, `register(route, handler)` y `guardsMiddleware[]`. Cada módulo exporta su función de renderizado.

US-19 Prioridad: Alta

Como desarrollador, **quiero** estructurar la aplicación en módulos independientes con separación clara de responsabilidades, **para** facilitar el mantenimiento, las pruebas y la escalabilidad vertical del proyecto.

Criterios de aceptación

- Cada módulo (auth, cursos, notas, gamificacion, comunicados) vive en su propia carpeta con su HTML parcial, JS y CSS.
- Los servicios de API (fetch) están centralizados en una capa /services separada del UI.
- El estado de sesión se gestiona en un módulo /store centralizado.

- No hay lógica de negocio en los archivos de vista (HTML parciales).

Escalabilidad vertical: La arquitectura modular permite escalar verticalmente añadiendo módulos nuevos (ej. mensajería, asistencia) sin refactorizar los existentes.

Nota técnica: Arquitectura sugerida: `/pages` (HTML parciales), `/services` (fetch API), `/store` (sesión), `/router` (enrutador), `/components` (elementos reutilizables UI).

Resumen de Historias de Usuario

ID	Rol	Historia (resumen)	Prioridad	Módulo
US-01	Estudiante / Rector / Profesor	Iniciar sesión con credenciales	Alta	Auth
US-02	Usuario autenticado	Mantener sesión al refrescar	Alta	Auth
US-03	Rector	Gestionar comunicados institucionales	Alta	Comunicados
US-04	Rector	Crear y gestionar cursos con horarios	Alta	Cursos
US-05	Rector	Gestionar registro de estudiantes	Alta	Usuarios
US-06	Rector	Gestionar registro de profesores	Alta	Usuarios
US-07	Rector	Consultar dashboard con indicadores	Media	Dashboard
US-08	Rector	Configurar tipos de punto (gamificación)	Media	Gamificación
US-09	Profesor	Ver cursos asignados	Alta	Cursos
US-10	Profesor	Registrar y publicar notas	Alta	Notas
US-11	Profesor	Asignar puntos de gamificación	Alta	Gamificación
US-12	Profesor	Ver horario y comunicados	Media	Comunicados
US-13	Estudiante	Consultar boletín de notas	Alta	Notas
US-14	Estudiante	Ver cursos y calendario	Alta	Cursos
US-15	Estudiante	Ver puntos y leaderboard	Alta	Gamificación
US-16	Estudiante	Canjear puntos en tienda	Media	Gamificación
US-17	Estudiante	Editar perfil personal	Baja	Perfil
US-18	Usuario	Navegar sin recarga (MPA router)	Alta	MPA
US-19	Desarrollador	Arquitectura modular escalable	Alta	MPA